Assignment 2, Part 3(a): CUBIC vs. Reno vs. BBR

Ayusha Patra Kabileswaran Krishna Kumar Vihaan Kodeboyina Millan Kumar

Summary

Across five public iperf3 servers and three repetitions, **BBR had the highest goodput in all 15 comparisons** (78.1 Mbit/s mean vs. 52.1 for CUBIC and 43.6 for Reno), but added 12–19 ms of queueing delay and retransmitted about 11% of its bytes. CUBIC beat Reno in 13 of 14 comparisons, by more on longer paths. The paths were *loss-limited*: sparse losses kept CUBIC and Reno at a roughly constant window, while BBR, which ignores loss, was not limited this way.

Setup

Host	Ubuntu 24.04.5 VM (Multipass) on an Apple Silicon Mac, Wi-Fi uplink
Kernel	Linux `6.8.0-147-generic` (aarch64), BBRv1 via `modprobe tcp_bbr`
Command	`python3 main.py -n 5 -d 60 -i 0.2 -r 3 --seed 42`
Tests	60 s each, TCP_INFO sampled every 0.2 s, 3 repetitions × 5 destinations × 3 algorithms = 45

Part 3 reuses the five destinations from Part 1 (minimum RTT in parentheses): LeaseWeb US (48 ms), FortiSASE ES (105 ms), MilkyWan FR (109 ms), Multinet24 PL (130 ms), and Mauritius Telecom (288 ms). In each repetition, every destination runs all three algorithms back to back, and the order rotates: CUBIC→Reno→BBR, then Reno→BBR→CUBIC, then BBR→CUBIC→Reno. Each test is preceded by a 3 s pause, and refused tests are retried up to twice. One test (Reno, repetition 3, LeaseWeb) was refused every time, so 44 of 45 tests completed. The algorithm is set with `setsockopt(TCP_CONGESTION)` before connecting and verified with `getsockopt`.

Metrics (all from TCP_INFO on the data socket): goodput from Δ`bytes_acked`; smoothed RTT; queueing delay = mean RTT − `min_rtt`; loss proxy = retransmitted bytes / bytes sent, and retransmissions per interval in the plots.

Results

The plots use `148.230.45.213:30009`, the representative destination from Question 2, with shared axis scales for all three algorithms.

Analysis

Traces (Figure 1). Reno grows its window by about one segment per RTT and halves it on loss (one loss in about 40 s here). CUBIC shows its cubic sawtooth, dropping to 70% and regrowing within seconds. BBR keeps a 1,300–3,300 segment window and dips almost to zero at about 12, 31, and 48 s; these are its ProbeRTT phases. BBR is the only algorithm with sustained retransmissions (Figure 2).

Operating regime: loss-limited. On the four paths from 48 to 130 ms, CUBIC and Reno kept nearly the same mean window (530–610 segments), so goodput ≈ cwnd/RTT fell roughly as 1/RTT. A capacity limit would instead make the window track each path's BDP. A constant window matches

Table 1: Per destination: mean ± std. dev. of per-run goodput across repetitions, and means of the other metrics.

Destination (min RTT)	Algo	Goodput (Mbit/s)	RTT (ms)	Queueing (ms)	Retrans. bytes	cwnd (segments)
LeaseWeb US (48 ms)	CUBIC	93.2 ± 1.4	77	28.7	0.04%	601
	Reno	85.7 ± 3.7	72	25.6	0.36%	538
	BBR	**144.5** ± 11.9	91	44.5	12.64%	2838
FortiSASE ES (105 ms)	CUBIC	48.6 ± 1.4	133	25.6	0.07%	588
	Reno	45.0 ± 1.6	129	23.3	1.01%	544
	BBR	**67.5** ± 1.4	141	35.7	9.08%	1961
MilkyWan FR (109 ms)	CUBIC	47.1 ± 2.0	132	24.1	0.05%	571
	Reno	45.0 ± 1.4	132	24.4	0.66%	546
	BBR	**72.8** ± 6.2	142	36.0	15.51%	2327
Multinet24 PL (130 ms)	CUBIC	41.5 ± 3.9	153	22.3	0.52%	605
	Reno	36.0 ± 2.1	155	23.8	0.54%	529
	BBR	**63.3** ± 5.3	168	37.5	11.43%	2360
Mauritius Tel. (288 ms)	CUBIC	30.0 ± 5.5	321	27.8	0.51%	834
	Reno	20.2 ± 2.5	316	22.4	1.00%	565
	BBR	**42.4** ± 1.9	332	39.3	7.39%	3207

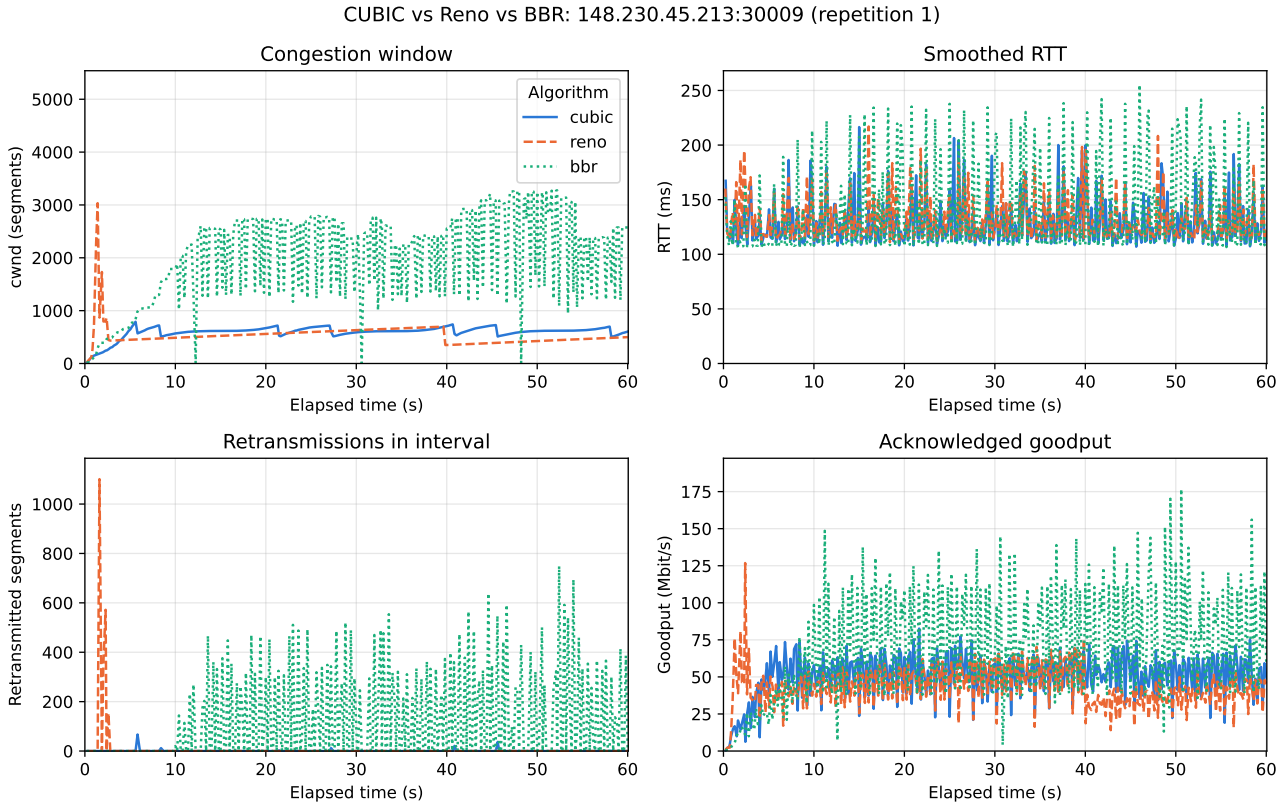


Figure 1: Time series at the representative destination (repetition 1).

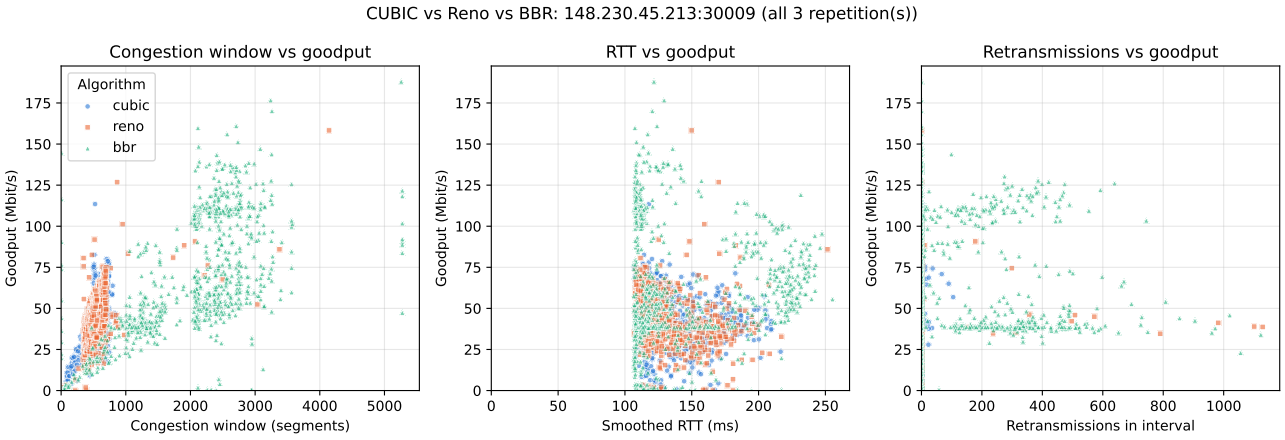


Figure 2: cwnd, RTT, and retransmissions vs. goodput (all 0.2 s samples, all three repetitions).

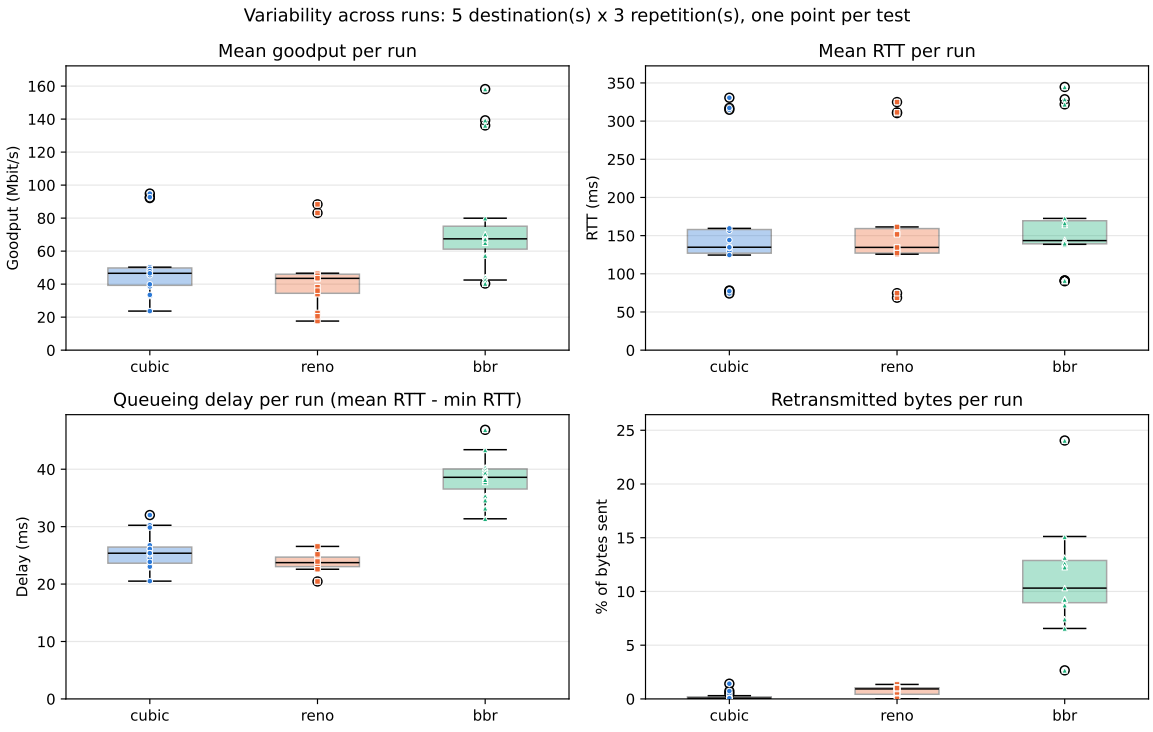


Figure 3: Run-to-run variability, one point per test (all destinations).

the AIMD throughput model, cwnd $\approx 1.22/\sqrt{p}$: Reno's one loss per ∼155,000 segments ($p \approx 6 \times 10^{-6}$) predicts about 480 segments, close to the 530–565 measured. BBR does not back off on loss, so these sparse losses did not limit it.

Which algorithm is better where.

- **Throughput: BBR**, by 1.5–1.8× Reno on the 48–130 ms paths and 2.1× at 288 ms. On a long path, a loss-based algorithm needs more round trips to regrow after each backoff. BBR suits bulk transfer on high-BDP paths with sparse non-congestive loss.
- **CUBIC vs. Reno: CUBIC on high-BDP paths.** The gap was 1.05–1.15× up to 130 ms but 1.48× at 288 ms (window 834 vs. 565). CUBIC backs off less (to 70%, not 50%), and its growth depends on time since the last loss rather than on RTT, whereas Reno would need over 2 minutes to regrow 500 segments at 288 ms.
- **Delay and loss: CUBIC and Reno.** Both had about 22–28 ms of queueing delay on every path. Because this floor is the same everywhere, it is probably jitter from our Wi-Fi and VM networking. BBR added 12–19 ms above it and retransmitted 7–16% of its bytes: BBRv1 keeps up to 2× the estimated BDP in flight and probes at 1.25× its bandwidth estimate, so it repeatedly overfills the bottleneck buffer.
- **BDP.** The BDP implied by achieved goodput is about 0.5–0.9 MB on the shorter paths and up to 1.5 MB at 288 ms. CUBIC and Reno's ∼0.8 MB windows are near it on short paths but well below it on the long one, where they trail BBR most. BBR's 2.8–4.6 MB windows exceed the BDP, and the excess becomes queueing delay and loss.

We did not observe a deep-buffer regime, where BBR should have the *lower* RTT, or a short uncongested path, where all three should reach link capacity.

Variability. On the same destination, results were consistent: the coefficient of variation of per-run goodput was 1.5–9.4% for 13 of 15 destination–algorithm pairs, and up to 18% on the long Mauritius path (Figure 3). Across destinations, goodput varied 3–4×, so the path matters more than the repetition. The ranking held regardless of test order: BBR was fastest in all 15 groups, and CUBIC beat Reno in 13 of 14 (the exception was a 0.9 Mbit/s difference).

Limitations. Public paths change between tests, and three repetitions is a small sample. We tested one flow at a time, so we could not measure fairness or BBR's effect on competing flows. The bottleneck location and buffer size are unknown, and our VM on Wi-Fi adds jitter. BDP is a lower bound because true capacity is unknown. TCP_INFO sampling every 0.2 s averages short bursts. Results apply to BBRv1 only; later versions react to loss.

Reproduction and code

From `Assignment2/` on Linux with `reno cubic bbr` available:

```
python3 main.py -n 5 -d 60 -i 0.2 -r 3 --seed 42
```

Raw traces go to `output/part3/`. The summaries, logs, and plots used here are in `Assignment2/part3a-results/`.

- Experiment loop (Part 1 destinations, rotating order, retries): `run_part3`; per-run metrics: `path_metrics`; CLI options: `main.py` L45–96.

- Algorithm selection and check: `run_test.py` L150–170; goodput and loss proxy: `compute_sample`.
- Plots: `plot_comparison` (per-algorithm pages are also in `part3a-results/comparison.pdf`).

Use of AI tools

We used Claude Code (Anthropic) to help set up the Linux VM, implement the Part 3 changes, and draft this writeup from our measurement files. We reviewed the code, tested it with short runs before the full experiment, and checked the numbers against `part3a-results/`.